Python Pandas:-

*Pandas is a **Python** library used for working with data sets. It has functions for*

1) Analysing,
2) Cleaning,
3) Exploring, and
4) Manipulating data.

*The name "Pandas" has a reference to both "Panel Data", and "Python Data Analysis" and was created by **Wes McKinney** in 2008.*

Pandas allow us to analyse big data and make conclusions based on statistical theories. Pandas can clean messy data sets, and make them readable and relevant. Relevant data is very important in data science.

Usually we work on two type of datasets

1) ***Series:***
2) ***Dataframe:***

Series:

*A pandas Series is a **one-dimensional** labelled array capable of holding any data type (e.g., integers, floats, strings, etc.). It is similar to a Python list or NumPy array but provides additional functionalities and features. Each element in a Series is associated with an index, which labels the elements and allows for efficient data retrieval and manipulation.*

```
import pandas as pd
# Series in Pandas
a = [1, 7, 2]
myvar = pd.Series(a)
print(myvar)
```

```
0    1
1    7
2    2
dtype: int64
```

```
# Series in Pandas

calories = {"day1": 420, "day2": 380, "day3": 390}
myvar = pd.Series(calories)
print(myvar)
```

```
day1    420
day2    380
day3    390
dtype: int64
```

DataFrame:

*In pandas, a DataFrame is a **two-dimensional** labeled data structure, similar to a table or **spreadsheet**. It consists of **rows and columns**, where each column can contain data of different types (e.g., integers, floats, strings, etc.). DataFrames are incredibly versatile and are widely used for data manipulation, analysis, and exploration in Python.*

```
# Create Dataframe

data = {
  "calories": [420, 380, 390],
  "duration": [50, 40, 45]}

#Load data into a DataFrame object:
df = pd.DataFrame(data)
print(df)
```

```
   calories  duration
0       420        50
1       380        40
2       390        45
```

Different Operation of DataFrame: -

1) ***Reading Dataframe***
2) ***Creating Dataframe***
3) *Filtering Dataframes:*
4) *Handling the missing Values*
   - *fillna*
   - *dropna*
5) *Dropping Row or Columns*
6) *Working on Index*
7) *iterate over data frame*
8) *Group by*
9) *Pivot*
10) *Joining Dataframe*
    - *Concatenation*
    - *Merge*
    - *Join*
11) *Some stats Function (measure of central tendency)*
    - *Mean*
    - *Median*
    - *Mode*
12) *Reshaping*
13) *Basic pandas Functions/Attributes*

Reading and Saving DataFrame

Data Input:

1. read_csv() : Read a CSV file into a DataFrame.
2. read_excel() : Read an Excel file into a DataFrame.
3. read_json() : Read a JSON file into a DataFrame.
4. read_sql() : Read from a SQL database into a DataFrame.

Save file

1. to_csv() : Write a DataFrame to a CSV file.
2. to_excel() : Write a DataFrame to an Excel file.
3. to_json() : Write a DataFrame to a JSON file.
4. to_sql() : Write records stored in a DataFrame to a SQL database.

Filtering Dataframes:

Below is the list of ways you can filter the datasets

1) *Indexing and slicing*
2) *.loc*
3) *.iloc*
4) *Conditional filtering*

indexing or slicing:

Bleow will print all the rows starting from 2 and ending at 4th index No.

```
# print all the row starting from 2 ending at 4th index No.
new_df[2:5]
```

	First_name	Last_name	Age	City
2	Tina	Ali	36	Pune
3	Jeetu	Gandhi	21	Delhi
4	Meera	Kumari	23	Bihar

loc:

It is primarily label-based and is used for selecting rows and columns by labels or a boolean array.

*df.loc[**rowNumber**,[**ColumnName**]]*

df.loc[2:4,["Firstname","LastName"]]

at:

*It is label-based and is used for accessing a **single value** in a DataFrame by row and column labels.*
dff.at[rowNumber,columnName]
df.at[2,"FirstName"]

---

iloc:

It is primarily integer position-based and is used for selecting rows and columns by integer position or a boolean array.
df.iloc[rowNumber,[columnNumber]]
df.iloc[2:4,[1,3]

iat:

It is integer position-based and is used for accessing a single value in a DataFrame by row and column integer positions.
df.iat[rowNumber,columnNumber]
df.iat[2,3]

Filtering using condition

1) *Below will print temp which is greater than 32*

```
1 new_df[new_df["Age"]>32]
```

	First_name	Last_name	Age	City
0	Ram	Kumar	42	
1	Mohan	Sharma	52	
2	Tina	Ali	36	Pune

2) *This will print only the **"Age"** and **"Last_name"** columns where the temperature is greater than 32.*

```
1 new_df.loc[new_df["Age"]>32,["First_name","Last_name"]]
```

	First_name	Last_name
0	Ram	Kumar
1	Mohan	Sharma
2	Tina	Ali

```
1 new_df.iloc[new_df[new_df["Age"]>32].index,[1,2]]
2
```

	Last_name	Age
0	Kumar	42
1	Sharma	52
2	Ali	36

Handling the missing Values
Use of fillna and dropna
df.fillna(value=None, method=None, axis=None, inplace=False)
Use of fillna and dropna
df.dropna(axis=0, how='any', subset=None, inplace=False)

This will replace NA values with 0.
new_df = df.fillna(0)
Copy paste earlier cell value to the empty cell
new_df = df.fillna(method="ffill")
Copy paste below cell value to the empty cell
new_df = df.fillna(method="bfill")
Copy paste next column cell value to the empty cell
new_df = df.fillna(method="bfill",axis= "column")
Copy and paste the earlier cell value to the next empty cell, however, this will only happen for one empty cell.
new_df = df.fillna(method="ffill",limit=1)
replace empty cells with a specific value:
new_df = df.fillna(130, inplace = True)
Replace NULL values in the "Calories" columns with the number 130:
new_df = df["Calories"].fillna(130,inplace = True)
Remove all rows containing NA
new_df = df.dropna()
This will keep those rows that have some or the other data and delete only those data that are competently empty.
new_df = df.dropna(how="all")
This will delete those rows which have two empty cells
new_df = df.dropna(thresh=2)
remove all rows containing NULL
df.dropna(inplace = True)
This will return True or False if the data frame contains duplicate
print(df.duplicated())
To remove the duplicate line
df.drop_duplicates(inplace = True)

Setting Index
This will set day column as index of the data-Frame
df.set_index('day',inpace=True)
This will reset original index of the Data-Frame
df.reset_index(inplace=True)
Reseting index
new_df.reset_index(drop=True,inplace=True)

Basic of pandas

Print First 5 Lines	df.head()
Print Last 5 Lines	df.tail()
Info about data	df.info()
How to print all the name of the column.	df.columns
How to print data type of column	Type(df['Name'])
Print max value of the column	df.['temperature'].max()
Print min value of the column	df.['temperature'].min()
Print min value of the standard deviation	df.['temperature'].std()
This will print statistics	df.describe

Different ways to iterate over data frame:

- ***.Iterows()***
- ***.Itertupples()***
- ***.Items()***
- ***Using index***
- ***Appy()***
-

.Iterows()

```
'''Using Iterrows'''
for index, row in df.iterrows():
    print(index, row)
    print()
```

```
0 First_name          Ram
Last_name          Kumar
Age                   42
City              Mumbai
Qulification       B.Com
Name: 0, dtype: object

1 First_name         Mohan
Last_name          Sharma
Age                   52
City               Noida
Qulification         IAS
Name: 1, dtype: object
```

.Itertupples()

```
'''Using itertuples'''
for i in df.itertuples():
    print(i)
```

```
Pandas(Index=0, First_name='Ram', Last_name='Kumar', Age=42, City='Mumbai', Qulification='B.Com')
Pandas(Index=1, First_name='Mohan', Last_name='Sharma', Age=52, City='Noida', Qulification='IAS')
Pandas(Index=2, First_name='Jeetu', Last_name='Ali', Age=36, City='Pune', Qulification='LLB')
Pandas(Index=3, First_name='Jeetendra', Last_name='Gandhi', Age=21, City='Delhi', Qulification='B.Tec
Pandas(Index=4, First_name='Meera', Last_name='Kumari', Age=23, City='Bihar', Qulification='MBBS')
```

.Items()

```
'''Using items'''
for colNmae, colValue in df.items():
    print(colNmae)
    print(colValue)
    print()
```

```
First_name
0          Ram
1        Mohan
2        Jeetu
3    Jeetendra
4        Meera
Name: First_name, dtype: object

Last_name
0     Kumar
1    Sharma
2       Ali
3    Gandhi
4    Kumari
Name: Last_name, dtype: object
```

Group by

df.groupby(by=None, axis=0, level=None, as_index=True, sort=True, group_keys=True, squeeze=<object object>, observed=False, dropna=True)

new_df = df.groupby('city')

This will group the data as per city.

```
for city in  city_df in new_df:
        print(city)
print(city_df)
```

This will print the data with group.

new_df.get_group('mumbai')

Concatenate: -

pd.concat(objs, axis=0, join='outer', ignore_index=False, keys=None, levels=None, names=None, verify_integrity=False, sort=False, copy=True)

- Paste data one below the other(appending the data)

new_df = pd.concat(df1,df2,ignore_index=True)

Paste data next to each other

new_df = pd.concat(df1,df2,ignore_index=True,axis=1)

Merge

```
pd.merge(right, how='inner', on=None, left_on=None, right_on=None, left_index=False, right_index=False, sort=False, suffixes=('_x', '_y'), copy=True, indicator=False, validate=None)
```

Create Data-frame with inner join

```
df3 = pd.merge(pd1,df2,on='City')
```

Create Data-frame with outer join

```
df3 = pd.merge(pd1,df2,on='City',how="outer")
```

Create Data-frame with left join

```
df3 = pd.merge(pd1,df2,on='City',how="left")
```

Pivot

pd.pivot(data, index=None, columns=None, values=None)

pd.pivot_table(data, values=None, index=None, columns=None, aggfunc='mean', fill_value=None, margins=False, dropna=True, margins_name='All', observed=False)

This will create pivit table with city as row and date in column	**new_df =pd.pivot_table(index= "city", column= "date", aggfunc= "sum")**

Reshaping the data:

df.melt(id_vars=None, value_vars=None, var_name=None, value_name='value', col_level=None, ignore_index=True)

This wll update the column data one bleow the other	**new_df = pd.melt(df,id_vars= ["day"])**

df.stack(level=-1, dropna=True)

This will create stacked data	**new_df = df.stack(level=0)**

Pandas uses the **mean()** **median()** and **mode()** methods to calculate the respective values for a specified column:

mean()	x = df["Calories"].mean() df["Calories"].fillna(x, inplace = True)
median()	x = df["Calories"].median() df["Calories"].fillna(x, inplace = True)

mode()	x = df["Calories"].mode()[0] df["Calories"].fillna(x, inplace = True)

Convert Data Frame into a Correct Format:

Convert all cells in the 'Date' column into dates.	import pandas as pd df = pd.read_csv('data.csv') df['Date'] = pd.to_datetime(df['Date']) print(df.to_string())

Data Exploration:

5. head(), tail() : View the first/last n rows of a DataFrame.
6. info() : Print a concise summary of a DataFrame.
7. describe() : Generate descriptive statistics of a DataFrame.
8. value_counts(): Count unique values in a Series.
9. unique(), nunique(): Get unique values/count of unique values in a Series.
10. isnull(), notnull(): Check for missing values.
11. corr(), cov() : Compute pairwise correlation/covariance of columns.

Data Manipulation:

12. groupby(): Group DataFrame using a mapper or by a Series of columns.
13. pivot_table(): Create a spreadsheet-style pivot table.
14. stack(), unstack(): Pivot a level of the (possibly hierarchical) index labels.
15. melt(): Unpivot DataFrame from wide format to long format.
16. drop(): Drop specified labels from rows or columns.
17. fillna(): Fill missing values.
18. dropna(): Drop missing values.
19. rename(): Alter axes labels.
20. map(), apply(), applymap(): Apply functions to elements of DataFrame or Series.
21. replace(): Replace values in a DataFrame or Series.

astype(): Cast a pandas object to a specified dtype.

Sorting:

22. sort_values(), sort_index(): Sort DataFrame by values or index.

Time Series:

23. date_range(), period_range(): Generate date and time ranges.
24. to_datetime(): Convert arguments to datetime.
25. resample(): Convenience method for frequency conversion and resampling.

These are just some of the many functions provided by pandas for data manipulation and analysis.

String Handling:

Accessors:
str: Accessor for vectorized string operations. Used to access string methods on Pandas Series.

Counting and Length:
str.count(): Count occurrences of a pattern/substring in each string.
str.len(): Compute the length of each string.

Substitution and Replacement:
str.replace(): Replace occurrences of a pattern/substring with another string.

String Searching and Matching:
str.contains(): Check if pattern or regex is contained within a string.
str.startswith(), str.endswith(): Check if string starts/ends with a specified substring.

Splitting and Joining:
str.split(), str.rsplit(): Split strings into substrings by a delimiter.
str.join(): Join lists contained as elements in the Series with a separator.
str.cat(): Concatenate strings.

String Slicing and Indexing:
str.slice(), str.slice_replace(): Slice substrings from each element in the Series/Index.
str.get(): Get the ith element of each string.
str.slice_replace(): Replace the ith element of each string with another string.

String Formatting:
str.format(): Format strings.
Case Conversion:
str.lower(), str.upper(), str.title(): Convert case of strings.

Stripping and Padding:
str.strip(), str.lstrip(), str.rstrip(): Remove leading/trailing whitespace.
str.pad(): Pad strings.

Alliging Text:
str.center(), str.ljust(), str.rjust(): Center, left-align, or right-align strings.
Use of Regex:
str.extract(), str.extractall(): Extract groups from the first match of a regular expression pattern or from all matches.
str.match(): Determine if each string matches a regular expression.
str.findall(): Find all occurrences of pattern or regular expression in the Series/Index.